CS571: Programming Languages
Assignment 2. Solution

Problem 1 [10pt] Consider the following class instances in a C++ program:

```
1  static myClass* X;
2  int main()
3  {
4    myClass* Y = new myClass();
5    foo();
6    delete Y;
7    return 0;
8  }
9
10 void foo()
11 {
12   myClass Z;
13   X = new myClass();
14 }
```

a) (5pt) What is the storage allocation (static/stack/heap) for the following objects?

- object A: the pointer `X`
- object B: the pointer `Y`
- object C: the `myClass` object created at line 4
- object D: the `myClass` object created at line 12
- object E: the `myClass` object created at line 13

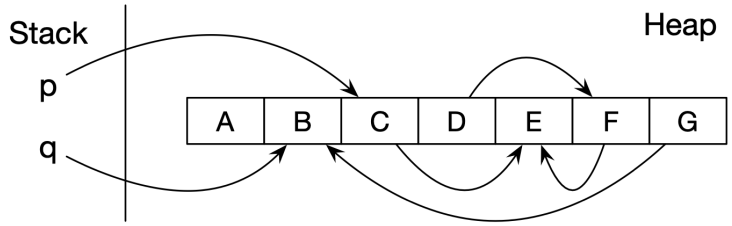
Solution:
A: static area, B: stack allocated, C: heap allocated, D: stack allocated, E: heap allocated.

b) (5pt) Consider one execution of the program above. The execution trace (a sequence of program statements executed at run time) of this program is `4 5 12 13 6 7`.
For each object above (i.e., object A to E), write down its lifetime (use a subset of execution trace, e.g., `12 13` to represent the lifetime). Assume the lifetime of a heap-allocated object starts from `new` and ends after `delete` and include both `new` and `delete` .

Solution:

Lifetime of A: `4 5 12 13 6 7`
Lifetime of B: `4 5 12 13 6 7`
Lifetime of C: `4 5 12 13 6`
Lifetime of D: `12 13`
Lifetime of E: `13 6 7`

Problem 2 [8pt] Consider the heap state as shown below before garbage collection:



a) (2pt) For the Mark-and-Compact algorithm, what objects remain on the heap after collection? Do they stay in the same location after collection?

Solution:
Objects B, C, E remain on the heap after collection. No.

b) (6pt) Consider algorithms Mark-and-Sweep, Mark-and-Compact, Stop-and-Copy. For each of them, write down the objects being visited during the collection in sequence. Assume 1) reachability analysis uses depth-first search, which will skip objects that have already been visited, 2) search starts from p, and then q, 3) when the entire heap is traversed, objects are visited from left to right. You don't need to write down the newly created object copies, if any.

Solution:
Mark-and-Sweep: C, E, B (Mark), A, B, C, D, E, F, G (Sweep).
Mark-and-Compact: C, E, B (Mark), A, B, C, D, E, F, G (Compute new addresses), A, B, C, D, E, F, G (Move objects to new address).
Stop-and-Copy: C, E, B.

Problem 3 [6pt] For each of the following data characteristics, **briefly explain** which of the garbage collection algorithms discussed in class (Mark-and-Sweep, Mark-and-Compact, Stop-and-Copy) would be the most appropriate and why. Make no other assumptions than those given.

a) (3pt) All objects are of the same size and many have relatively long lifetime.

Solution:
Mark-and-Sweep is the best since most objects have relatively long lifetime, and it avoids the cost of copying data. Moreover, since objects are of the same size, it does not introduce fragmentation.

b) (3pt) Most objects have short lifetime and data locality greatly improves application's performance.

Solution:
Stop-and-Copy is suitable for data with short lifetime. It also eliminates fragmentation, and hence, improves data locality.

Problem 4 [14pt] Consider the following pseudo-code, with the assumption that the language uses **static scoping** and has one scope for each function (including the main function), and it allows nested functions (hence, nested scopes).

```
1  main() {
2    int g;
3    int x = 4;
4    function B(int a) {
5      int x = a + 2;
6      C(1);
7    }
8    function A(int n) {
9      g = n;
10   }
11   function C(int m) {
12     print x;
13     x = x / 2;
14     if (x > 1)
15       C(m + 1);
16     else
17       A(m);
18   }
19   // body of main
20   B(1);
21   print g;
22 }
```

a) (5pt) Draw the symbol table tree with 4 tables, one table for each of the 4 scopes (namely `main`, `B`, `A`, `C`) in this program. Assume each table contains two columns: name and kind (e.g, id, fun, para).

Solution:
`main` (Table M):

Name	Kind
g	id
x	id
B	fun
A	fun
C	fun

B (Table B):

Name	Kind
a	para
x	id

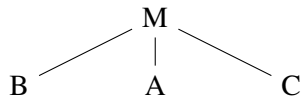
A (Table A):

Name	Kind
n	para

C (Table C):

Name	Kind
m	para

The hierarchy of tables is as follows:



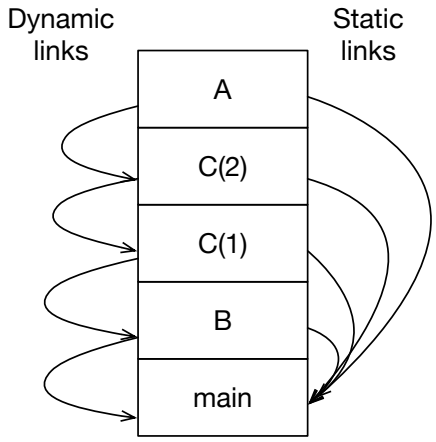
b) (3pt) What does the program print?

Solution:
4 2 2

c) (4pt) Draw a picture of the run-time stack when function `A` has just been called. For each frame, show the static and dynamic links. You do not have to show the storage for any other information.

Solution: see the picture below.

d) (2pt) Refer to the run-time stack, briefly explain how function `A` finds variable `g`.



Solution: It uses the static links to locate the frame of Main. Within that frame, it finds `g` which is at a statically known offset.

Problem 5 [12pt] Consider the following pseudo-code with higher-order function support. Assume that the language has one scope for each function, and it allows nested functions (hence, nested scopes). In the pseudo code, declarations with type `int` are local declarations; otherwise the variables not declared locally are referring to the outer level of function.

```
function A() {
  int x = 5;
  function C(P) {
    int x = 3;
    P();
  }
  function D() {
    print x;
  }
  function B() {
    int x = 4;
    C(D);
  }
  B();
}
A();
```

a) (3pt) What would the program print if this language uses dynamic scoping and shallow binding (binding happens when the function is called)? **Solution**: 3

b) (6pt) Justify your answer by showing the tree of symbol tables when execution reaches the `print` expression.

Solution:
Function `A` (Table A):

Name	Kind
x	id
C	fun
D	fun
B	fun

Function `C` (Table C):

Name	Kind
P	para
x	id

Function `D` (Table D):

Name	Kind

Function `B` (Table B):

Name	Kind
x	id

The symbol table tree when the print statement is executed:

A
|
B
|
C
|
D

So the program will print out the `x` in function `C`, which is 3.

c) (3pt) What would the program print if this language uses static/lexical scoping?

Solution: 5

In static scoping the variable `x` in function `D` will always refer to the local variable `int x = 5` in function `A` that `D` is nested in.